Junior Developers Can Build with AI Coding Agents

A case study of the WorxForge internship at ITWorx

Hany Gheit
Senior Software Engineering Manager, ITWorx
Mentor of the WorxForge internship team
Summer 2026

Publication version 1.0 · October 12, 2026

Executive summary

Four computer science students built and demonstrated WorxForge during a four week ITWorx summer internship. Most began with basic C# knowledge, and their frontend experience ranged from little to fair. With a concept-focused learning plan, GitHub Copilot, and senior mentorship, they delivered an application combining an ASP.NET Core API and a React frontend. According to my account as their mentor, the main implementation phase took one week. [1]

I had encountered the view that working effectively with AI coding agents requires senior developers and leaves little room for juniors. This project gave us a practical example of students contributing to software delivery with AI assistance. I guided the team, but I did not write code or make commits for the first version they presented. [1]

The interns progressed through learning, requirements planning, implementation, testing, and a local demonstration. Their final presentation reported work across database design, backend and frontend development, unit and integration testing, and peer collaboration. [2,4]

The case supports a specific conclusion: students with limited experience in the chosen stack can build a working application with AI tools and mentorship. It also suggests that concept-focused preparation can bring learners into practical project work earlier. The evidence does not establish a measured productivity multiplier, equivalent learning outcomes to a longer program, or production readiness.

For engineering leaders developing junior talent, the opportunity is to give learners meaningful work while strengthening their ability to explain, test, and maintain what they build. The WorxForge experience offers an example of that approach and a basis for more systematic evaluation.

Team background and product scope

Undergraduate students learning a new stack

The team comprised four computer science students from different universities and faculties, at different stages of their undergraduate studies. Most had basic knowledge of C#, with limited practical experience in React. These are descriptions from my account of the cohort, rather than results from a standardized skills assessment. [1]

Their curiosity and willingness to learn stood out to me. I guided the learning approach, helped frame the project, and supported decisions as the work progressed. The interns implemented the application and delivered the demonstration.

A knowledge asset platform

WorxForge was designed to help people publish, discover, and reuse knowledge assets such as prompts, agent instructions, workflows, templates, and coding standards. The project gave the interns connected user workflows to build across the interface, API, and data layers. [3]

The planned scope included asset metadata and content, discovery and filtering, saved libraries, profiles, collaboration, and notifications. Account and administration capabilities were also included in the requirements. The requirements defined the intended scope; they are not evidence that every listed capability was completed. [3]

The application combined an ASP.NET Core backend with a React frontend. The team's presentation also described React with TypeScript, clean architecture, dependency injection, and the repository pattern as part of their learning and project experience. [4]

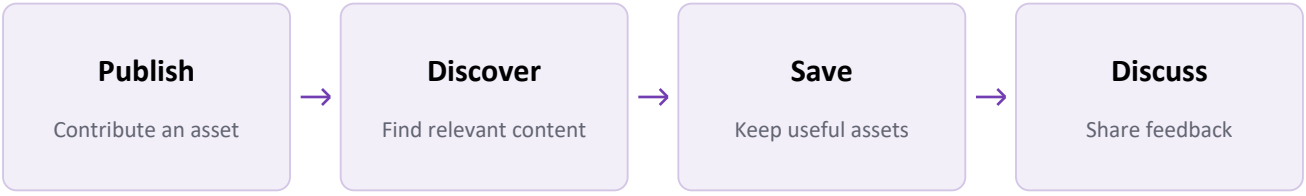


Figure 1. A schematic of the intended knowledge asset workflow. It illustrates the product concept and is not a screenshot or a verification of feature completeness.

At the final local demonstration, the application presented a consistent interface across discovery, publishing, saved assets, and activity notifications. The demonstration and supplied interface views support that observation; they do not independently establish reliability or security. [1,5]

The four week learning and delivery plan

The internship allocated one week each to initial learning, requirements and planning, main implementation, and testing and deployment. This sequence reflects my account of the project. The written plan also proposed earlier integration work during planning and quality activities throughout implementation. [1,2]

Week	Primary focus	Mentor reported outcome
1	Concepts and guided self study	Initial preparation completed in about one week. Some interns used an additional day.
2	Requirements and planning	Product requirements, repository preparation, and an implementation plan.
3	Implementation with GitHub Copilot	Main implementation phase completed in one week.
4	Testing and deployment	A locally deployed application presented on the final day.

Concepts before memorization

In earlier internship programs, we had allocated at least a month to a broadly similar learning plan. For this cohort, we changed the emphasis. We asked the interns to focus on concepts and the big picture, with less time spent memorizing syntax or repeating implementation exercises. Most completed the initial learning phase within a week; some needed one additional day. [1]

The preparation used mostly free educational resources, including YouTube material. Topics included C# and .NET fundamentals, data access with Entity Framework Core, the ASP.NET Core request lifecycle, frontend prerequisites, React components and hooks, and GitHub Copilot. Supporting material covered architecture, unit testing, and version control. [1,2]

The aim was to help learners understand what the code should do and how its parts communicate, so they could guide implementation with AI assistance. Reading code, recognizing errors, and understanding behavior remained part of the learning task.

The plan included an explain-and-defend habit and proposed tasks with AI disabled to assess understanding. The case does not provide assessment scores or evidence that every planned assessment was completed. Those planned activities should therefore be distinguished from the reported delivery results. [2]

Planning implementation and demonstration

Requirements gave the team a shared target

During the second week, the interns used AI assistants to help develop requirements and plan the application. They prepared the repository and the implementation approach. The project requirements gave them a shared description of the product and its workflows. [1,3]

Publishing a knowledge asset, for example, involves content, metadata, validation, persistence, and how the asset appears in discovery. Planning those relationships gives a learner a basis for directing a coding tool and assessing whether the resulting behavior meets the requirement.

The interns owned the initial implementation

The main implementation phase took place in the third week using GitHub Copilot. In this case, agentic coding refers to the interns guiding AI tools through implementation and iteration. The available evidence does not quantify how much code was generated or which level of automation was used. [1]

Their final presentation described work from requirements through database design, backend, and frontend implementation. It also reported unit and integration testing, Git and GitHub collaboration, peer code review, and sprint planning. No independent historical test execution results or coverage measurements are established by this paper. [4]

For the version presented at the end of the internship, I did not write code, make commits, or inspect the source. My involvement was mentorship. The interns owned that implementation. Senior guidance in this case should therefore not be interpreted as a comprehensive senior code review. [1]

A working application in a local setting

The final week focused on deployment and testing in a controlled local environment. On the last day, the interns presented the deployed application. In that demonstration, the agreed workflows behaved as expected, and I was impressed by the consistency and quality of the interface. [1]

That was a meaningful delivery result for this cohort. The demonstration showed an application running in its local setting. Establishing production readiness would require further evidence, including security review, broader testing, and operational validation.

Findings and evidence limits

What the case supports

Four students with limited experience in the selected stack delivered a working local application during a four week internship. My account places the initial learning phase at about one week and the main implementation phase at one week. The team used AI tools and retained responsibility for the first version they demonstrated. [1]

Evidence basis	Supported claim and boundary
Mentor account	Cohort background, reported schedule, mentor involvement, and observations from the final demonstration.
Project plan and requirements	Intended learning approach, delivery phases, and product scope. They describe plans and requirements.
Team presentation	Reported experience with architecture, testing, peer review, and implementation across application layers.
Demo interface views	Visible product screens and interface consistency. They do not independently verify complete workflows.

Earlier entry into practical work

Compared with my experience of earlier learning programs, this cohort reached practical project work sooner. Concept-focused preparation and AI assistance appeared to help them progress quickly, alongside mentorship, motivation, and a defined scope. [1]

The comparison concerns different programs and different learners. The case does not isolate the effect of AI or provide comparable effort records and learning assessments. It supports an observation of a shorter initial preparation period followed by useful delivery, without establishing a general speed improvement.

A bounded observational case

This paper concerns one cohort and one application. It does not provide a controlled comparison, long-term maintenance results, defect-rate comparisons, or measured retention of skills. It also does not establish that every requirement was completed or that the internship application was ready for production.

The result gives a concrete reason to question the assumption that junior developers cannot contribute with AI coding agents. Repeating the approach with more cohorts and clearer measurements would help establish how consistently it works and how much mentorship is required.

Implications for junior development programs

Combine early contribution with understanding

The experience suggests an approach for engineering managers, mentors, and educators: give learners a bounded product task, support implementation with AI, and examine both the delivered behavior and the learner's understanding. The following practices are recommendations for repeating and strengthening the approach; they are not additional claims about completed activities in this cohort.

1. **Tie concepts to the project.** Explain the request lifecycle, data flow, state, validation, and failure behavior through the application the learners will build. Use AI to support questions and examples, while checking the learner's explanation.

2. **Define observable acceptance criteria.** Agree on user workflows and expected results before implementation. A connected workflow helps learners understand the relationship between the interface, API, and data.

3. **Work in manageable increments.** Ask learners to explain a change, test it against the requirement, and discuss it with a peer. Evaluate the reasoning behind a change alongside its visible behavior.

4. **Assign senior review where it matters.** Review authorization, sensitive data handling, external services, and deployment behavior before relying on the application. This would strengthen the assurance process beyond the mentorship and demonstration reported here.

5. **Assess understanding independently.** Include explain, debug, and modify tasks with AI disabled, as the learning plan proposed. Record results so delivery speed can be evaluated alongside learning. [2]

Measure delivery and learning together

A future evaluation should record time to the first reviewed contribution, acceptance-test results, mentor effort, and defects found after integration. Independent explanation and debugging tasks would help assess understanding. Follow-up maintenance work would help show whether learners retained the skills.

The same approach may inform graduate development programs, but this cohort comprised undergraduate students. It does not directly measure outcomes for fresh graduates, all junior developers, or unmentored learners.

The opportunity is to bring learners into meaningful work earlier while helping them develop engineering judgment. The WorxForge case shows that AI assistance can be part of that process.

WorxForge as an open-source project

WorxForge is publicly available under the MIT license:

github.com/itworx/WorxForge

The internship application and the public release are separate milestones. Release preparation included configuration cleanup, consolidation of the frontend and backend into a monorepo, and changes to onboarding and maintenance. Those later changes should be distinguished from what the interns built and demonstrated. [1,6]

The public version reviewed on October 12, 2026 uses a shared guest workspace without per-user isolation. Its documented capabilities include asset discovery and publishing, bookmarks, comments, reactions, profiles, and in-app notifications. Account flows require further work, email delivery is disabled, and some administration capabilities remain unfinished. These limitations describe the public version, distinct from the historical demonstration. [6]

Contributions are welcome to complete account and administration flows, strengthen testing and accessibility, improve discovery, and make setup and maintenance clearer. Contributors should consult the repository documentation for the current feature status as the project evolves.

Acknowledgments

The four interns built the original application. Their commitment and willingness to learn made the experiment possible. I was proud to see them present a working application at the end of the internship.

WorxForge offers a concrete example of students building with AI coding agents and mentorship. For teams developing junior talent, it is a useful case to examine and an approach worth evaluating further.

Evidence references

[1] Hany Gheit. Firsthand mentor account of the Summer 2026 cohort, schedule, involvement, and final demonstration. The timing and background descriptions are reported observations.

[2] Internship learning and delivery plan. Concepts, explain-and-defend approach, quality activities, and proposed assessments. Planned activities are distinguished from verified outcomes.

[3] WorxForge project requirements. Product purpose, intended scope, and user workflows.

[4] Internship team presentation. Reported technical learning, testing, collaboration, and work across application layers.

[5] Internship demonstration interface views. Evidence of visible screens and layout consistency. Original images are not embedded in this publication version.

[6] WorxForge public repository README, feature-status documentation, and MIT license, reviewed October 12, 2026. Current source status is distinct from the original internship delivery.